



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

Department of Computer Engineering Generative AI Lab Manual AY 2026-27

Class: BTech(CE)
Course Name: Generative AI (PE III)
Faculty Name: Prof. Yogita Patil

Semester: VII
Course Code: CE413
Academic Year: 2026-27

Experiment 6: *Fine-Tuning and Comparative Evaluation of a Pretrained BERT Model for Sentence Classification*

1. Aim

To design, implement, fine-tune, and evaluate a pretrained BERT model for a real-world sentence classification task using Hugging Face Transformers, and analyze the effect of preprocessing, tokenization, classification-head architecture, hyperparameters, class imbalance, and input robustness on model performance.

2. Learning Objectives

After completing this experiment, students will be able to:

- Understand the architecture and working of BERT for sequence classification.
- Prepare and preprocess a text classification dataset.
- Perform BERT tokenization using Hugging Face.
- Fine-tune a pretrained BERT model for a classification task.
- Modify and compare classification-head architectures.
- Investigate the effect of hyperparameters and sequence length.
- Handle class imbalance using appropriate techniques.
- Evaluate classification performance using multiple metrics.
- Perform error and robustness analysis on unseen inputs.
- Draw conclusions from controlled experimental comparisons.

3. Problem Statement

Design, implement, and evaluate a BERT-based sentence classification system using Hugging Face Transformers for a selected real-world classification task such as sentiment analysis, spam detection, news classification, or intent classification. Develop an appropriate text preprocessing and BERT tokenization pipeline, fine-tune the pretrained BERT model, and systematically investigate the effect of sequence length, preprocessing strategy, classification-head architecture, and training hyperparameters on model performance. Further, analyze the impact of class imbalance and evaluate model robustness on unseen, noisy, and paraphrased inputs using suitable classification metrics, confusion matrices, and error analysis.



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

4. Software Requirements

1. Python 3.x
2. Jupyter Notebook / Google Colab
3. PyTorch
4. Hugging Face Transformers
5. Hugging Face Datasets
6. scikit-learn
7. pandas
8. NumPy
9. Matplotlib
10. Execution Environment: Google Colab with GPU runtime is recommended. The experiment can also be executed on a local CPU with a reduced dataset and fewer training epochs.

Recommended Installation

```
!pip install transformers datasets evaluate accelerate scikit-learn
```

5. Dataset

Select one sentence-classification problem.

Domain	Example Task	Classes
Sentiment Analysis	Review classification	Positive / Negative
Spam Detection	SMS/email classification	Spam / Ham
News Classification	News categorization	Sports / Business / Politics etc.
Intent Classification	Customer/HR queries	Multiple intents

Dataset Requirements

The dataset should contain:

- ✓ Text/sentence column
- ✓ Class/label column
- ✓ Sufficient samples for training and testing
- ✓ Clearly defined class labels

Divide the dataset into: Training → Validation → Testing

A typical split may be: 70% → 15% → 15%



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

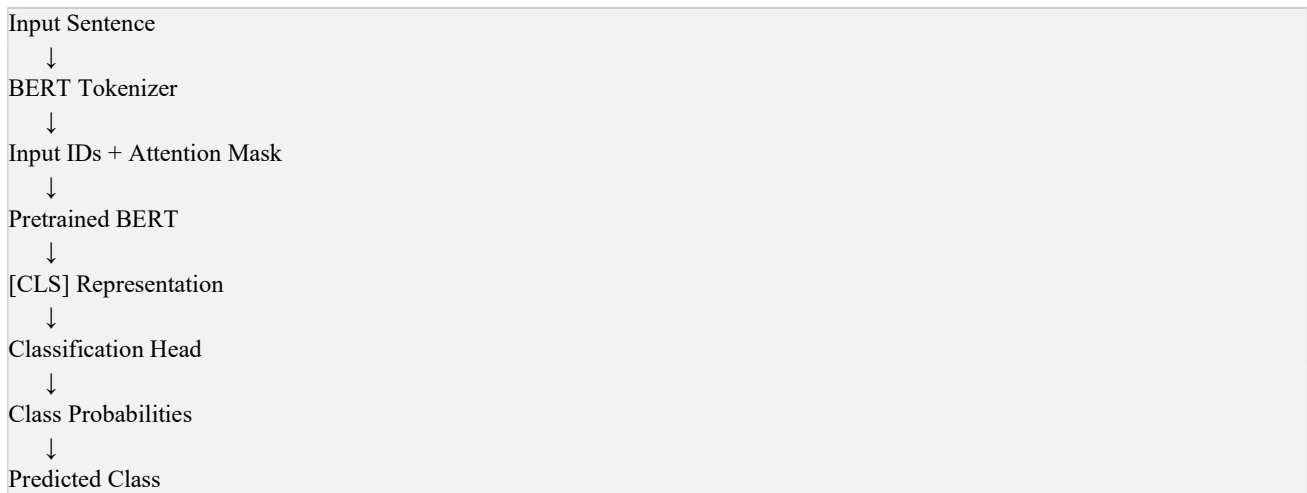
6. Background Theory

6.1 What is BERT?

BERT – Bidirectional Encoder Representations from Transformers – is a pretrained Transformer-based language model.

Unlike traditional sequential models, BERT uses self-attention to understand relationships between words based on their surrounding context.

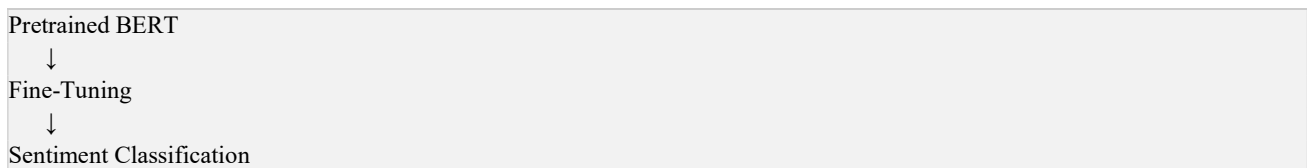
For classification:



6.2 Why Fine-Tune BERT?

BERT is initially pretrained on large text corpora.

Fine-tuning adapts the pretrained knowledge to a specific downstream task. For example:



The pretrained model provides general language representations, while fine-tuning adjusts them for the selected classification task.

6.3 BERT Tokenization

BERT uses WordPiece tokenization. A sentence is converted into tokens and then numerical IDs.

Special tokens include:

- ✓ [CLS] – beginning/classification representation
- ✓ [SEP] – separates or terminates sequences



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

- ✓ [PAD] – padding
- ✓ [MASK] – used during pretraining

The tokenizer also generates an attention mask indicating which positions contain actual input tokens.

7. Procedure

Task 1 – Dataset Exploration and Preprocessing

- a) Load the selected dataset.
- b) Display sample records.
- c) Identify missing values.
- d) Remove duplicate records.
- e) Examine class distribution.
- f) Check noisy or inconsistent text.
- g) Divide the dataset into training, validation, and test sets.

Experiment

Create two versions:

- ✓ Version A: Minimal preprocessing
- ✓ Version B: Cleaned/normalized text

Compare their classification performance later.

Observation: Determine whether extensive traditional NLP preprocessing is actually beneficial for BERT.

Task 2 – BERT Tokenization Analysis

Load a pretrained BERT tokenizer.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "bert-base-uncased"
)
```

Tokenize sample sentences. Examine:

- ✓ Tokens
- ✓ Token IDs
- ✓ Attention masks
- ✓ Padding
- ✓ Truncation

Example:



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

```
text = "I really enjoyed this movie."
```

```
encoded = tokenizer(  
    text,  
    padding="max_length",  
    truncation=True,  
    max_length=32  
)
```

```
print(encoded)
```

Experiment

Repeat the experiment using at least two sequence lengths:

- ✓ max_length = 32
- ✓ max_length = 128

Record the effect on:

- ✓ Number of tokens
- ✓ Training time
- ✓ Memory requirement
- ✓ Classification performance

Task 3 – Establish a BERT Baseline

Load a pretrained sequence-classification model.

```
from transformers import AutoModelForSequenceClassification  
  
model = AutoModelForSequenceClassification.from_pretrained(  
    "bert-base-uncased",  
    num_labels=2  
)
```

Implement the complete pipeline:

```
Dataset  
↓  
Tokenizer  
↓  
BERT  
↓  
Classification Head  
↓  
Prediction
```

Record the baseline performance.



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

Task 4 – Fine-Tune BERT

Fine-tune BERT using the training dataset. Experiment with at least three training parameters, such as:

Parameter	Values to Investigate
Learning Rate	2e-5, 3e-5, 5e-5
Batch Size	8, 16
Epochs	2, 3, 5
Sequence Length	32, 64, 128
Weight Decay	0, 0.01

Record:

- ✓ Training loss
- ✓ Validation loss
- ✓ Accuracy
- ✓ Precision
- ✓ Recall
- ✓ F1-score
- ✓ Training time

Important

Do not simply select the configuration giving the highest accuracy. Students must analyze why a particular configuration performs differently.

Task 5 – Classification Head Experiment

Compare different classification-head configurations.

Model A – Linear

BERT → Linear → Softmax

Model B – Dropout

BERT → Dropout → Linear → Softmax

Model C – Dense + Dropout

BERT → Dense → Dropout → Linear → Softmax



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

Compare the models using the same dataset and training conditions.

Analyze

- ✓ Accuracy
- ✓ F1-score
- ✓ Validation loss
- ✓ Overfitting
- ✓ Generalization

Task 6 – Class Imbalance Analysis

Analyze the class distribution. If the dataset is imbalanced, compare:

Approach A

Standard cross-entropy loss

Approach B

Class-weighted loss

Evaluate using:

- 1) Accuracy
- 2) Macro-F1
- 3) Precision
- 4) Recall
- 5) Confusion matrix

Key Question

Why can accuracy be misleading when the dataset contains imbalanced classes?

Task 7 – Error and Robustness Analysis

Evaluate the final model using unseen and challenging inputs. Create test examples involving:

1. Spelling errors
2. Abbreviations
3. Negation
4. Paraphrasing
5. Short sentences
6. Long sentences
7. Ambiguous sentences
8. Domain-specific terminology

Example:

Original: "I am extremely happy with the service."



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

Paraphrased: "The service exceeded my expectations."

Compare the predictions. Identify at least 10 incorrectly classified examples and categorize the errors.

Task 8 – Comparative Analysis

Prepare a final comparison.

Experiment	Configuration	Accuracy	Precision	Recall	F1
E1	Baseline BERT				
E2	Different sequence length				
E3	Best hyperparameters				
E4	Modified classifier head				
E5	Imbalance handling				

Also record:

Configuration	Training Time	Validation Loss	Generalization
Baseline			
Modified			
Final			

8. Required Plots

Students should generate at least:

Plot 1 Training Loss vs Epoch

Plot 2 Validation Loss vs Epoch

Plot 3 Accuracy/F1 vs Experiment

Plot 4 Confusion Matrix

These plots should be used to support the final analysis.



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Autonomous Institute Affiliated to University of Mumbai)

9. Results and Discussion

Students should answer the following:

- i. How does BERT tokenization differ from traditional NLP preprocessing?
- ii. What is the role of [CLS] in sentence classification?
- iii. How did sequence length affect performance and computational cost?
- iv. Which hyperparameter had the greatest effect on performance?
- v. Did modifying the classification head improve generalization?
- vi. How did class imbalance affect accuracy and F1-score?
- vii. Which errors occurred most frequently?
- viii. How robust was the model to spelling errors and paraphrased inputs?
- ix. Did the best-performing model also provide the best generalization?
- x. What are the major limitations of the developed BERT classifier?

10. Expected Outcome

Students should successfully:

- I. Implement a BERT-based sentence classifier.
- II. Fine-tune a pretrained BERT model.
- III. Compare multiple preprocessing/tokenization configurations.
- IV. Experiment with classification-head architectures.
- V. Analyze hyperparameter effects.
- VI. Handle class imbalance.
- VII. Evaluate the model using multiple metrics.
- VIII. Perform error and robustness analysis.
- IX. Justify the final model configuration using experimental evidence.

11. Conclusion

The experiment demonstrates how a pretrained BERT Transformer can be adapted for sentence classification and how different design choices — including preprocessing, sequence length, classification-head architecture, hyperparameters, and imbalance handling — affect model performance and generalization.

Students should conclude the experiment by identifying the most suitable configuration based on experimental evidence, rather than merely reporting the highest accuracy.